

MASTER DEVELOPMENT PROMPT — TAMAM

0. الدور المطلوب منك

أنت الآن تعمل كفريق هندسي متكامل يتكون من:

- Principal Software Architect
- Senior Backend Engineer
- Senior Flutter Engineer
- Senior Frontend Engineer
- DevOps Engineer
- Database Architect
- Cybersecurity Engineer
- QA Automation Engineer
- UX/UI Product Designer
- GIS / Maps Engineer
- Payments Engineer
- Product Manager

مهمتك هي تصميم وبناء منصة **TAMAM** كاملة وقابلة للإطلاق التجاري **Production-Ready**.

لا أريد Prototype. لا أريد Demo شكلي. لا أريد صفحات Frontend بدون Backend حقيقي. لا أريد بيانات وهمية داخل مسارات الإنتاج. لا أريد أزرارًا لا تعمل. لا أريد TODO أو FIXME أو Placeholder في الوظائف الأساسية. لا أريد معالجة مؤقتة للمشكلات.

أي Feature يتم إضافتها يجب أن تكون:

1. مرتبطة بالواجهة.
2. مرتبطة بال-Backend.
3. مرتبطة بقاعدة البيانات عند الحاجة.
4. محمية بالصلاحيات المناسبة.
5. خاضعة للتحقق Validation.
6. لديها معالجة أخطاء.
7. لديها Logging مناسب.
8. لديها اختبارات.
9. قابلة للاستخدام فعليًا.
10. موثقة.

1. اسم المشروع

اسم المنصة:

TAMAM

الاسم العربي:

تمام

نوع المشروع:

Multi-Service Mobility & Local Services Platform

المنصة تجمع في تطبيق واحد:

- نقل الركاب.
- توصيل الأغراض.
- الخدمات المنزلية والمهنية.
- الخدمات العاجلة.

مع إمكانية إضافة مستقبلاً:

- توصيل الطعام.
- المطاعم.
- البقالة.
- الصيدليات.
- المتاجر.
- التنظيف.
- نقل الأثاث.
- .Roadside Assistance
- خدمات B2B.
- خدمات عند الطلب .On-Demand Services

يجب بناء النظام من البداية بطريقة Modular تسمح بإضافة هذه الخدمات بدون إعادة كتابة النظام الأساسي.

2. فلسفة TAMAM

TAMAM ليست تطبيق Taxi فقط.

ولست تطبيق صيانة فقط.

ولست تطبيق توصيل فقط.

TAMAM عبارة عن:

Universal Local Services Platform

المستخدم يستطيع فتح تطبيق واحد وطلب:

سيارة.

أو:

شخص يوصل غرضًا.

أو:

سباك.

أو:

كهربائي.

أو:

فني صيانة.

أو أي خدمة تتم إضافتها مستقبلاً.

3. الخدمات المتاحة عند الإطلاق V1

يجب إطلاق TAMAM بثلاثة محاور رئيسية:

TAMAM Ride 3.1

خدمة نقل الركاب.

:Customer

.Pickup → Destination → Driver → Tracking → Trip → Payment → Rating

TAMAM Delivery 3.2

توصيل غرض من موقع إلى موقع.

.Pickup → Package → Courier → Tracking → Delivery → Proof of Delivery

TAMAM Services 3.3

طلب فني أو مقدم خدمة.

مثل:

- سباك.
- كهربائي.
- فني تكييف.
- فني أجهزة.
- أقفال.
- نجار.
- دهان.
- المنيوم.
- صيانة أجهزة.
- خدمات أخرى.

يجب أن يستطيع Admin إضافة أو تعديل أو تعطيل Service Categories بدون تعديل الكود.

4. التطبيقات المطلوبة

أنشئ النظام على أربعة أجزاء:

A. TAMAM Customer

:Flutter Application

.Android + iOS

B. TAMAM Partner

:Flutter Application

.Android + iOS

يستخدمه:

- Driver
- Courier
- Technician
- Service Provider

ويستطيع Partner امتلاك أكثر من Role.

مثال:

DRIVER
COURIER

أو:

PLUMBER
ELECTRICIAN

C. TAMAM Admin

.Responsive Web Application

يجب أن يعمل بكفاءة على:

- .Desktop
- .Laptop
- .Tablet

ويحتوي لوحة عمليات لحظية Real-Time Operations Dashboard.

D. TAMAM Backend

Backend مركزي يحتوي جميع Business Rules.

ممنوع الاعتماد على تطبيق الهاتف لحماية قواعد العمل أو حساب السعر أو تحديد صلاحية انتقال حالة الطلب.

Server هو:

Source of Truth

Technology Stack .5

اعتمد البنية التالية ما لم يوجد سبب هندسي قوي وموثق لتغييرها.

Mobile

.Flutter

:Architecture

.Clean Architecture + Feature-Based Modular Structure

:State Management

استخدم حلاً Production-grade مناسباً مثل Riverpod/BLoC مع اعتماد أسلوب واحد بشكل موحد.

Backend

.Node.js + TypeScript + NestJS

:Architecture

Modular Monolith

في الإصدار الأول.

لكن يجب فصل Domains داخلياً بحيث يمكن تحويل أي Module إلى Microservice مستقبلاً.

لا تبدأ بعشرات Microservices غير الضرورية.

Admin

.Next.js + TypeScript

.Responsive UI

Database

.PostgreSQL

مع:

.PostGIS

Cache + Geo + Distributed Locks

.Redis

Real Time

.WebSockets

مع Authentication و Authorization

Background Jobs

Queue System مثل:

.BullMQ + Redis

استخدمه في:

- .Notifications
- .Matching
- .Timeouts

- .Webhooks
 - .Scheduled jobs
 - .Media jobs
 - .Reports
 - .Retry operations
-

File Storage

.S3-Compatible Object Storage

لا تخزن ملفات الصور والفيديو داخل PostgreSQL.

.6 Repository Structure

يفضل Monorepo منظم مثل:

```
/tamam
/apps
/customer-mobile
/partner-mobile
/admin-web
/api
/packages
/shared-types
/validation
/ui-tokens
/config
/infrastructure
/docs
/scripts
/tests
```

يجب تجنب نسخ نفس DTOs والEnums بشكل يدوي بين المشاريع متى كان ذلك ممكناً بأمان.

Core Domain .7

لا تقم ببناء منطق منفصل بالكامل لكل:

Ride

Delivery

.Service

أنشئ Universal Job Engine.

الكائن المركزي:

Job

ويحتوي نوعًا:

RIDE

DELIVERY

HOME_SERVICE

وفي المستقبل:

FOOD

GROCERY

PHARMACY

SHOPPING

MOVING

ROAD_ASSISTANCE

Service Catalog .8

أنشئ:

ServiceType

ServiceCategory

ServiceSubcategory

ServiceOption

كل Service Category يجب أن تدعم Configuration وليس Hard Coding.

مثال:

HOME_SERVICE
Plumbing
Water Leak
Sink
Toilet
Pipes

ويستطيع Admin:

- إضافة Category.
- تعديلها.
- إضافة Icon.
- تغيير ترتيبها.
- إخفاؤها.
- تحديد المناطق المتاحة فيها.
- تحديد طريقة التسعير.
- تحديد الوثائق المطلوبة من مقدم الخدمة.
- تحديد هل تسمح بالحجز الفوري.
- تحديد هل تسمح بالحجز المجدول.

9. User System

أنشئ Entity موحدة:

User

مع Profiles مختلفة.

Roles:

CUSTOMER
PARTNER
ADMIN
SUPPORT
DISPATCHER
FINANCE

لا تستخدم Role واحدة فقط في الحقل إذا كان النظام يحتاج Multi-role.

أنشئ RBAC حقيقي.

10. Authentication

يجب دعم:

- .Phone Number Authentication
- .OTP
- .Refresh Token
- .Access Token
- .Device Sessions

ويجب تطبيق:

- .OTP expiration
- .OTP retry limits
- .OTP resend cooldown
- .Brute-force protection
- .Rate limiting
- .Device session management
- .Token rotation
- .Logout current device
- .Logout all devices
- .Revocation

يجب عدم تخزين Tokens الحساسة بطريقة غير آمنة في Mobile App.

11. Customer Profile

يحتوي على:

- .UUID
- .Full name

- .Phone
- .Email optional
- .Profile image
- .Default language
- .Default currency
- .Rating
- .Completed jobs
- .Cancelled jobs
- .Account status
- .Created at
- .Updated at

حالات الحساب:

ACTIVE
RESTRICTED
SUSPENDED
DELETED

12. Partner Profile

Partner يحتوي:

- .Personal information
- .Profile photo
- .Phone
- .Identity verification
- .Roles
- .Skills
- .Service categories
- .Rating
- .Number of jobs
- .Acceptance rate
- .Cancellation rate
- .Current availability
- .Verification status
- .Wallet
- .Documents
- .Service areas

:Verification Status

DRAFT
PENDING
UNDER_REVIEW
APPROVED
REJECTED
SUSPENDED

Partner Documents .13

أنشئ Document Verification System مرئاً.

أنواع محتملة:

- ID
- .Driving license
- .Vehicle license
- .Insurance
- .Professional certificate
- .Business document
- .Profile picture

لكل Document:

type
number
file
issuedAt
expiresAt
verificationStatus
verifiedBy
verifiedAt
rejectionReason

ويجب إرسال تنبيه قبل انتهاء الوثائق.

Vehicle System .14

:Entity

Vehicle

:Fields

- .Owner
- .Partner
- .Type
- .Brand
- .Model
- .Year
- .Color
- .Plate
- .Seats
- .Photos
- .Active status
- .Verification status

Vehicle Types يجب أن تكون Admin configurable.

مثلاً:

ECONOMY
FAMILY
PREMIUM
MOTORBIKE
DELIVERY_CAR

15. Customer Home

الشاشة الرئيسية يجب أن تكون بسيطة.

تحتوي:

- .Current Location
- .Search
- .Services

:Cards

مشوار 🚗
توصيل 📦
خدمات 🛠️
خدمة عاجلة 🚒

ثم:

- .Recent requests
- .Favorite locations
- .Promotions
- .Available services

16. Saved Places

دعم:

Home
Work
Custom

مع إمكانية حفظ عدة أماكن.

17. TAMAM Ride Flow

Step 1

اختيار Pickup.

استخدم GPS مع إمكانية تحريك Pin يدويًا.

Step 2

اختيار Destination.

Step 3

احسب:

- .Distance
- .Route
- .ETA
- .Estimated price

Step 4

اعرض أنواع المركبات المتاحة.

مثلاً:

Economy
Family
Premium

مع:

- .ETA
- .Estimated price
- .Capacity

Step 5

عند الضغط:

Request Ride

يتم إنشاء Job.

Ride States .18

استخدم State Machine واضحة:

DRAFT
REQUESTED
SEARCHING
ASSIGNED
PARTNER_EN_ROUTE
PARTNER_ARRIVED
WAITING_CUSTOMER
IN_PROGRESS
COMPLETED
CANCELLED
NO_PARTNER_AVAILABLE
DISPUTED

لا تسمح بتغيير Status عشوائياً.

حدد Explicit State Transitions.

مثلاً:

ASSIGNED -> IN_PROGRESS

غير مسموح مباشرة.

يجب المرور بالقواعد المناسبة.

كل انتقال Status يسجل Audit Event.

Dispatch Engine .19

أنشئ Dispatch Engine منفصل منطقياً.

عند وجود Job:

ابحث عن Partners الذين:

• Online.

- Available
 - Approved
 - لديهم Role مناسب
 - لديهم Service المناسب
 - داخل Service Zone
 - لديهم Vehicle مناسب عند الحاجة
 - غير محظورين
 - ليس لديهم Job متعارض
-

20. Driver Matching

احسب Candidate Score باستخدام عناصر مثل:

- ETA
- Distance
- Partner rating
- Acceptance rate
- Cancellation rate
- Current workload
- Vehicle compatibility
- Service eligibility

لا تجعل Rating وحده هو العامل المسيطر.

21. Dispatch Waves

لا ترسل الطلب إلى مئات السائقين مباشرة.

استخدم Waves.

مثال قابل للتعديل:

Wave 1:

أقرب عدد محدد من السائقين.

ثم إذا لم يقبل أحد:

Wave 2.

ثم:

.Wave 3

Radius يمكن توسيعه تدريجيًا.

كل هذه الإعدادات Admin-configurable.

22. Concurrency Protection

مهم جدًا:

يجب منع قبول Driver لنفس Job.

استخدم:

- Atomic Database Transaction
- Row locking أو Optimistic Concurrency
- Distributed lock إذا لزم.

بعد إسناد الطلب:

أي محاولة قبول أخرى يجب أن تفشل بشكل آمن.

23. Live Tracking

عندما يكون Partner داخل Job نشط:

يرسل:

latitude
longitude
accuracy
heading
speed
timestamp

إلى Backend.

Backend يتحقق من البيانات ثم يبيث Location إلى الأشخاص المصرح لهم فقط.
لا تبث GPS العام لجميع المستخدمين.

24. Tracking Frequency

لا تستخدم إرسال GPS كل ثانية بدون سبب.

استخدم Adaptive Tracking.

مثلاً:

- Higher frequency أثناء المهمة.
- Lower frequency عند Online بدون Job.
- Stop عند Offline.

يجب مراعاة:

- Battery.
 - Network.
 - Data consumption.
 - Background execution limitations.
-

25. Tracking Security

يجب:

- Authenticate socket.
- Authorize subscription.
- Validate Job membership.
- Reject stale coordinates.
- Detect impossible jumps قدر الإمكان.
- Rate-limit location updates.

احفظ Tracking History للطلبات حسب سياسة Retention محددة.

ETA .26

اعرض:

- Partner ETA to pickup
- Trip ETA
- Remaining distance

لا تعتمد فقط على المسافة الخطية.

Driver Arrival .27

استخدم Geofence تقريبيًا مع إمكانية تأكيد السائق.

لا تعتبر السائق وصل إذا كان بعيدًا بشكل غير منطقي إلا بوجود Override موثق.

Ride Start Verification .28

يدعم النظام إمكانية:

.Trip PIN / OTP

مثلًا يحصل العميل على:

4821

ويبدأ السائق الرحلة بعد إدخال الرمز عند تفعيل هذه الخاصية.

Ride Completion .29

عند الوصول:

احسب Fare النهائي حسب Pricing Policy.

ثم:

- .Complete job
- .Process payment
- .Update wallet
- .Update metrics
- .Request rating
- .Generate receipt

كل هذه العملية يجب أن تكون Transaction-safe قدر الإمكان.

30. TAMAM Delivery Flow

يحدد العميل:

- .Pickup location
 - .Destination
 - .Sender details
 - .Recipient details
 - .Package category
 - .Approximate size
 - .Approximate weight
 - .Photos
 - .Description
 - .Delivery notes
-

31. Package Categories

.Admin configurable

مثال:

- .Documents
- .Small package
- .Medium package
- .Large package
- .Fragile

يجب دعم قائمة المواد المحظورة حسب سياسة المنصة مستقبلاً.

32. Pickup Verification

عند استلام الغرض يمكن استخدام:

Pickup OTP

أو إثبات آخر حسب Configuration.

33. Proof of Delivery

عند التسليم يدعم النظام:

- .Delivery OTP
- .Receiver name
- .Delivery photo
- .Timestamp
- .GPS
- .Signature optional

يجب حفظ Proof of Delivery داخل Job.

34. Multi-stop Architecture

حتى إذا لم يتم تفعيلها في V1، صمم Database بحيث Job يمكن أن يحتوي:

JobStops

بدل Pickup و Destination فقط.

للسماح مستقبلاً بـ:

.A → B → C → D

TAMAM Services .35

Customer يختار:

.Service Category

مثلاً:

Plumbing

ثم Subcategory.

Service Request Form .36

يسمح للمستخدم بإرسال:

- .Description
 - .Multiple images
 - .Video
 - .Audio
 - .Location
 - .Urgency
 - .Preferred date
 - .Preferred time
 - .Additional instructions
-

Immediate vs Scheduled .37

يدعم:

NOW
SCHEDULED

:NOW

البحث عن أقرب مقدم خدمة مناسب.

:SCHEDULED

اختيار تاريخ ووقت.

يجب منع Double Booking.

38. Urgent Services

يدعم:

STANDARD

URGENT

EMERGENCY

لكن أسماء ومستويات الخدمة قابلة للتعديل من الإدارة.

يمكن فرض Surcharge قابل للتكوين.

39. Technician Matching

يجب مراعاة:

- Skill
- Category
- Subcategory
- Service Zone
- Availability
- Rating
- Distance
- Documents
- Schedule
- Previous performance

Service Workflow .40

:State Machine

REQUESTED
SEARCHING
PROVIDER_ASSIGNED
PROVIDER_EN_ROUTE
PROVIDER_ARRIVED
INSPECTION_STARTED
QUOTE_REQUIRED
QUOTE_SUBMITTED
QUOTE_APPROVED
QUOTE_REJECTED
WORK_STARTED
WAITING_FOR_PARTS
WORK_COMPLETED
CUSTOMER_CONFIRMED
COMPLETED
CANCELLED
DISPUTED

حسب نوع الخدمة يمكن تجاوز بعض المراحل بواسطة Workflow Configuration.

Inspection Fee .41

ادعم:

Visit / Inspection Fee

يستطيع Admin تحديده:

- Global
- حسب المنطقة
- حسب Category
- حسب Provider Type

Quote System .42

الفني يستطيع بعد الفحص إنشاء Quote.

Quote يحتوي:

laborCost
partsCost
additionalFees
discount
tax
total
description
estimatedDuration

مع إمكانية إدخال عدة Items.

Quote Approval .43

Customer يحصل على:

Approve
Reject

ممنوع بدء العمل الذي يتطلب Quote قبل Approval.

كل تعديل Quote ينشئ Revision جديدًا.

لا تقم بتغيير Quote قديم بدون History.

Additional Work .44

إذا اكتشف الفني عملاً إضافيًا:

ينشئ:

Change Order

ويحتاج موافقة العميل قبل إضافته للسعر.

هذه النقطة مهمة لمنع النزاعات.

Pricing Engine .45

أنشئ Pricing Module مستقل.

لا تضع أسعارًا ثابتة داخل Mobile App.

كل التسعير Server-side.

Ride Pricing .46

يدعم:

Base Fare

Distance Rate

Time Rate

Minimum Fare

Booking Fee

Zone Fee

Service Fee

Surge Multiplier

Discount

Promo

Tax

Delivery Pricing .47

يدعم:

Base
Distance
Package Type
Weight
Vehicle Type
Urgency
Zone
Additional Stops

Home Service Pricing .48

يدعم:

- .Fixed price
- .Inspection + Quote
- .Hourly
- .Starting-from
- .Custom quote

بحسب .Category

Price Snapshot .49

عندما يبدأ Job:

احفظ نسخة من Pricing Rules المستخدمة.

لا تجعل تغيير Admin للأسعار يؤثر بآثر رجعي على طلب قائم.

Money Rules .50

ممنوع استخدام Floating Point للأموال.

استخدم:

• Integer Minor Units

أو طريقة Decimal آمنة.

مع:

currency_code

مثل:

ILS

USD

JOD

ولا تفترض Currency واحدة داخل منطق النظام.

51. Payments

ابن Payment Abstraction Layer.

يدعم:

CASH

WALLET

CARD

BANK

EXTERNAL_GATEWAY

حتى لو كانت بعض الطرق Disabled في البداية.

52. Payment Gateway

يجب عدم ربط Business Logic مباشرة بمزود دفع محدد.

استخدم:

PaymentProvider interface

حتى يمكن إضافة أكثر من Gateway.

53. Payment Idempotency

أي عملية مالية حساسة يجب دعم Idempotency.

خصوصًا:

- .Payment creation
- .Payment capture
- .Refund
- .Wallet credit
- .Withdrawal

يجب ألا يؤدي Retry إلى خصم المال مرتين.

54. Webhooks

Webhooks يجب:

- .Verify signature
 - .Validate payload
 - .Store event ID
 - .Prevent duplicate processing
 - .Retry safely
 - .Log failures
-

55. Wallet

أنشئ Wallet لكل:

Customer عند الحاجة.

.Partner

.Platform accounting

Ledger .56

ممنوع الاكتفاء بحقل:

balance

وتعديله مباشرة.

يجب وجود Immutable Financial Ledger.

كل حركة مالية تسجل.

يفضل نظام Double-Entry Ledger للأجزاء المالية الجوهرية.

مثال:

Trip 100

Platform Commission 15

Partner Earnings 85

يجب أن يمكن إعادة حساب الرصيد من Ledger.

Partner Earnings .57

Dashboard للشريك تعرض:

- .Today
- .Week
- .Month
- .Completed jobs
- .Gross earnings

- .Commission
 - .Bonuses
 - .Adjustments
 - .Net earnings
 - .Withdrawals
 - .Current balance
-

58. Commission Engine

Admin يستطيع تحديد Commission:

- .Global
- حسب Job Type
- .Category
- .Zone
- .Partner
- .Campaign

احتفظ بتاريخ صلاحية كل Policy.

59. Cancellation Rules

أنشئ Cancellation Policy Engine.

يعتمد على:

- نوع الطلب.
- مرحلة الطلب.
- الوقت منذ قبول الطلب.
- وصول مقدم الخدمة.
- .Customer/Partner
- .Zone

ويحسب Cancellation Fee إذا كانت السياسة تتطلب.

Promo Codes .60

يدعم:

- .Percentage
 - .Fixed amount
 - .Max discount
 - .Minimum order
 - .Start/end time
 - .Usage limit
 - .Per-user limit
 - .First-order only
 - .Specific services
 - .Specific zones
 - .Specific users
-

Referral System .61

كل مستخدم يمكن أن يمتلك Referral Code.

يدعم:

- .Reward inviter
 - .Reward new customer
 - .Conditions
 - .Fraud controls
 - .Expiration
-

Notifications .62

Notification Service موحد.

:Channels

- .Push
- .In-app
- .SMS abstraction

• Email abstraction

Events مثل:

JOB_ACCEPTED
PARTNER_ARRIVING
PARTNER_ARRIVED
JOB_STARTED
QUOTE_RECEIVED
QUOTE_APPROVED
JOB_COMPLETED
PAYMENT_SUCCESS
PAYMENT_FAILED

63. Notification Templates

لا تضع النصوص داخل Business Logic.

استخدم Templates.

وتدعم:

- .Arabic
- .English

ومستقبلاً لغات إضافية.

64. Chat

أنشئ Chat مرتبطاً بالJob.

Customer ↔ Assigned Partner

يدعم:

- .Text
- .Image
- .Location optional
- .Delivery status

- Read status

مع:

- Authentication
- Authorization
- Rate limiting

ولا يسمح لأي مستخدم عشوائي بالدخول إلى Chat.

65. Contact Privacy

صمم النظام بحيث يمكن إضافة Phone Number Masking مستقبلاً.

تجنب كشف المعلومات الشخصية بدون ضرورة.

66. SOS & Safety

للرحلات النشطة:

أضف:

SOS

و:

Share Trip

Share Trip يولد Link آمن محدود الصلاحية يسمح برؤية معلومات الرحلة الضرورية فقط.

67. Safety Center

Customer يمكنه:

- الإبلاغ عن Partner.
- الإبلاغ عن مشكلة.
- مشاركة الرحلة.
- الوصول إلى الدعم.

Partner أيضًا يستطيع الإبلاغ عن Customer.

68. Ratings

بعد Job مكتمل:

Customer → Partner

ويمكن:

Partner → Customer

:Rating

5–1.

مع Tags اختيارية.

69. Rating Integrity

لا تسمح بتقييم Job غير مكتمل.

تقييم واحد لكل طرف لكل Job.

يمكن تعديل التقييم خلال Window محدد فقط حسب السياسة.

70. Support Tickets

أنشئ Support System.

:Ticket

category
priority
customer
partner
job
status
assignedAgent
description
attachments

:Statuses

OPEN
IN_PROGRESS
WAITING_USER
RESOLVED
CLOSED

Disputes .71

Dispute مرتبط بـ Job.

يشمل:

- Reason
- Evidence
- Messages
- Payment impact
- Admin decision
- Refund
- Partner adjustment

مع Audit Trail

Refund System .72

Partial أو Full Refund

مع:

- .Reason
- .Actor
- .Payment transaction reference
- .Ledger entries

لا تسمح بعمليات Refund عشوائية بدون Permission.

73. Service Zones

استخدم PostGIS.

Admin يستطيع رسم Polygon على الخريطة يمثل:

Service Zone

ويحدد:

- .Available services
 - .Prices
 - .Operating hours
 - .Vehicle types
 - .Partner eligibility
-

74. Zone Validation

عند إنشاء Job:

Backend يتحقق من أن الموقع داخل Zone مدعومة.

لا تعتمد على Mobile فقط.

75. Operating Hours

لكل Service Zone أو Category:

- .Working days
- .Opening
- .Closing
- .Holidays
- .Exceptions

Admin Dashboard .76

يجب إنشاء Dashboard احترافية تعرض:

- .Active jobs
- .Searching jobs
- .Completed today
- .Cancelled today
- .Online partners
- .Available partners
- .Gross bookings
- .Platform revenue
- .Support issues

Live Operations Map .77

الخريطة تعرض حسب الصلاحية:

- .Online partners
- .Active jobs
- .Pickup
- .Destinations
- .Service zones

مع Filters.

لا ترسل كل المواقع لجميع موظفي الإدارة افتراضياً.

طبق Least Privilege.

78. Dispatcher Console

Dispatcher يستطيع:

- مشاهدة Unassigned jobs.
- البحث عن Partners.
- Manual assign.
- Reassign.
- Cancel وفق Permission.
- التواصل مع الأطراف.
- مشاهدة Timeline.

كل Manual Override يسجل في Audit Log.

79. Customer Management

Admin يستطيع:

- Search.
- View profile.
- View jobs.
- View payments.
- View support.
- Restrict.
- Suspend.

لا تستخدم Hard Delete للبيانات التشغيلية المهمة.

80. Partner Management

Admin يستطيع:

- Review documents.
- Approve.
- Reject.
- Suspend.
- Manage skills.

- .Manage vehicle
 - .Manage service zones
 - .View earnings
 - .View complaints
 - .View performance
-

81. Pricing Admin

أنشئ واجهة Pricing Configuration واضحة.

ولا تجعل الإدارة تضطر لتعديل Database يدويًا.

82. Service Management

Admin يستطيع إنشاء Service جديدة من الواجهة.

حدد:

- .Name AR
 - .Name EN
 - .Description
 - .Icon
 - .Job type
 - .Pricing method
 - .Required fields
 - .Required media
 - .Urgency support
 - .Scheduled support
 - .Active zones
-

83. Feature Flags

أنشئ Feature Flag System.

مثلاً:

```
food_module=false
card_payments=false
trip_pin=true
urgent_services=true
```

حتى يمكن إطلاق Features تدريجيًا.

84. Config Management

قيم مثل:

- .Search radius
- .Dispatch timeout
- .Cancellation grace period
- .Tracking intervals
- .Fees

يجب أن تكون Configurable.

لكن يجب تحديد:

- .Safe minimum
- .Safe maximum
- .Validation

85. Audit Log

كل عملية إدارية مهمة تسجل:

```
actor
action
entity
entityId
oldValue
newValue
timestamp
ip
```

لا تسمح للمستخدم الإداري بتعديل Audit Logs.

86. Security Requirements

تعامل مع المشروع على أنه يحتوي:

- بيانات شخصية.
- بيانات موقع.
- بيانات مالية.
- وثائق هوية.

لذلك الأمن شرط أساسي.

87. Backend Security

طبق:

- Strong authentication
 - RBAC
 - Object-level authorization
 - Input validation
 - Output filtering
 - Rate limiting
 - CSRF protection حيث ينطبق.
 - Secure CORS
 - Security headers
 - SQL injection prevention
 - XSS prevention
 - SSRF controls
 - Secure file uploads
 - Request size limits
 - Brute force protection
-

IDOR Protection .88

لا يكفي:

GET /jobs/:id

يجب التحقق أن المستخدم يملك صلاحية رؤية Job المحدد.

طبق Object Level Authorization في كل مورد حساس.

Secrets .89

ممنوع:

- API Keys داخل Git.
- Database passwords داخل source.
- Private keys داخل Mobile.
- Secrets داخل frontend.

استخدم Environment / Secrets Manager.

Password/OTP Logs .90

ممنوع تسجيل:

- .OTP
- .Password
- .Full tokens
- .Card data
- .Sensitive secrets

في Logs.

91. PII

صنف البيانات الحساسة.

طبق:

- .Encryption in transit
 - .Encryption at rest where appropriate
 - .Minimal collection
 - .Access control
 - .Retention policy
-

92. Location Privacy

GPS تعتبر بيانات حساسة.

لا تخزن Tracking إلى الأبد بدون ضرورة.

أنشئ Retention Policy قابلة للتعديل حسب المتطلبات القانونية والتشغيلية.

93. Secure Uploads

Uploads يجب:

- .MIME validation
- .Extension validation
- .Size limit
- .Generated filenames
- .Private buckets by default
- .Signed URLs
- .Malware scanning integration-ready

لا تثق باسم الملف القادم من المستخدم.

Media Metadata .94

قم بإزالة Metadata الحساسة مثل EXIF عندما يكون ذلك مناسباً للصور التي يعاد تقديمها للمستخدمين.

Database .95

استخدم UUID مناسب للـ IDs العامة.

ضع:

- Foreign keys
- Constraints
- Unique constraints
- Indexes

ولا تعتمد فقط على Validation في التطبيق.

Database Transactions .96

استخدم Transaction في العمليات التي تحتاج Atomicity.

مثل:

- Job assignment
 - Payment
 - Wallet transfer
 - Job completion
 - Refund
-

Database Migrations .97

كل تغيير Schema يجب أن يكون Migration.

ممنوع تعديل Production DB يدوياً كطريقة تشغيل عادية.

Soft Delete .98

استخدم Soft Delete فقط حيث يكون منطقيًا.

لا تطبق Soft Delete عشوائيًا على كل Table.

البيانات المالية و Audit يجب أن تتبع قواعد Immutable مناسبة.

Indexing .99

راجع Query Plans للعمليات المهمة.

خصوصًا:

- .Nearby partner
 - .Jobs by status
 - .Jobs by customer
 - .Jobs by partner
 - .Payments
 - .Audit search
-

API Design .100

استخدم API Versioning.

مثلًا:

/api/v1/

وفر OpenAPI documentation.

API Errors .101

استخدم Error Format موحداً مثل:

code
message
details
requestId

ولا ترسل Stack Trace إلى المستخدم.

Idempotency .102

الـ Endpoints الحساسة يجب أن تدعم Idempotency Key عند الحاجة.

مثل:

POST /jobs
POST /payments
POST /refunds

لمنع duplicate requests.

Optimistic Concurrency .103

في Entities الحساسة أضف Version أو طريقة تمنع Lost Updates.

خصوصاً:

- .Job
- .Quote
- .Payment state

Mobile Offline Handling .104

إذا انقطع الإنترنت:

- لا يتعطل التطبيق.
 - اعرض حالة واضحة.
 - Retry آمن.
 - لا تكرر عمليات مالية أو Job creation.
-

Connectivity Recovery .105

عند عودة الإنترنت:

.Sync state from server

لا تفترض أن الحالة المحلية ما زالت صحيحة.

Deep Links .106

جهاز Architecture لـ:

- .Job link
 - .Referral link
 - .Shared trip
 - .Promotions
-

Localization .107

التطبيق يجب أن يدعم من البداية:

Arabic RTL

و:

English LTR

ولا تقم بكتابة Text ثابت داخل Widgets.

استخدم Localization Files.

Arabic UX .108

اختبر:

- .RTL
- الأرقام.
- النصوص الطويلة.
- أسماء المدن.
- .Currency
- .Date/time

تأكد أن جميع الصفحات تعمل في RTL دون مشاكل Layout.

Currency .109

Currency ليست Hardcoded.

اعرض التنسيق حسب Locale.

واحفظ داخليًا بـ ISO Currency Code.

Time .110

خزن timestamps في UTC.

اعرضها حسب Time Zone الصحيح للمستخدم.

111. Accessibility

طبق:

- مناسب لأحجام الخط.
 - Contrast.
 - Touch targets.
 - Screen reader labels.
 - Logical navigation.
-

112. Performance

ضع Performance Budget.

تجنب:

- تحميل Lists ضخمة.
- الصور الأصلية عالية الحجم.
- Queries N+1.
- إعادة بناء Flutter widgets بلا داع.

استخدم Pagination.

113. Image Optimization

أنشئ:

- Thumbnail.
- Medium.
- Original إذا لزم.

مع Compression مناسب.

114. Observability

طبق:

- .Structured logs
 - .Metrics
 - .Distributed request ID
 - .Error tracking
 - .Health checks
-

Health Endpoints .115

مثل:

health/live/
health/ready/

مع عدم كشف معلومات حساسة.

Metrics .116

راقب:

- .API latency
 - .Error rate
 - .Queue delay
 - .DB connections
 - .WebSocket connections
 - .Job creation
 - .Dispatch success
 - .Payment failures
-

Product Metrics .117

أنشئ Analytics Events مثل:

app_opened
service_selected
job_created
partner_assigned
job_started
job_completed
job_cancelled
quote_approved
payment_success

بدون إرسال بيانات شخصية أكثر من الضرورة.

Business KPIs .118

لوحة Analytics يجب أن تدعم مستقبلاً:

- .Jobs/day
- .GMV
- .Platform revenue
- .Completion rate
- .Cancellation rate
- .Average dispatch time
- .Average pickup ETA
- .Active users
- .Repeat customers
- .Partner utilization

Backup .119

ضع استراتيجية Backup.

تشمل:

- .PostgreSQL
- .Critical configuration
- .Object storage metadata

مع اختبار Restore وليس Backup فقط.

Disaster Recovery .120

وثق:

- .Backup frequency
 - .Retention
 - .RPO target
 - .RTO target
 - .Restore procedure
-

Deployment .121

استخدم Docker.

أنشئ Environments منفصلة:

local
development
staging
production

Production لا تستخدم Development settings.

CI/CD .122

Pipeline يجب أن تنفذ:

lint
type-check
unit tests
integration tests
security checks

build

ثم Deployment controlled.

123. Production Deployment

يجب وجود:

- .HTTPS only
- .Secure secrets
- .Database migrations
- .Rollback strategy
- .Health check
- .Monitoring

124. Testing Strategy

لا تعتبر المشروع مكتملاً بدون Automated Tests.

أنشئ:

Unit Tests

.Business rules

Integration Tests

.Database + services

API Tests

.Endpoints

.Widget/Component Tests

E2E Tests

Mandatory E2E — Ride .125

اختبار:

.Customer creates Ride

.Driver gets request

.Driver accepts

.Customer sees driver

.Driver arrives

.Trip starts

.Trip completes

.Payment processed

.Customer rates

Mandatory E2E — Delivery .126

.Customer creates Delivery

.Courier accepts

.Pickup verified

.Courier moves

.Delivery OTP

.Order completed

Mandatory E2E — Home Service .127

.Customer uploads problem

.Technician accepts

.Arrives

.Creates quote

.Customer approves

.Work starts

.Work completes

.Customer confirms

.Payment/ledger complete

Race Condition Tests .128

اختبر:

Driver A و Driver B يحاولان قبول نفس الطلب في اللحظة نفسها.

النتيجة:

واحد فقط ينجح.

Payment Retry Tests .129

اختبر duplicate webhook.

يجب ألا يحدث:

.Double charge

.Double credit

Permission Tests .130

اختبر أن:

.Customer A لا يستطيع رؤية Customer B jobs

.Partner A لا يستطيع تعديل Partner B jobs

.Support لا يمتلك Finance permissions

Load Testing .131

اختبر سيناريوهات مثل:

- آلاف WebSocket connections
- عدد كبير من location updates
- Dispatch burst
- Notification burst

ووثق النتائج.

UX Error States .132

كل شاشة تعتمد على API يجب أن تمتلك:

- Loading
- Empty
- Error
- Retry
- Offline

لا تترك شاشة بيضاء.

App Lifecycle .133

تعامل مع:

- .App killed
- .Background
- .Resume
- .Token expiration
- .Location permission disabled
- .Notification permission disabled

Partner Availability .134

Partner يتحكم:

ONLINE
OFFLINE
BUSY

لكن Server هو الذي يقرر Final availability.

لا يمكن أن يكون Partner Available لطلبين متعارضين إذا كان Service لا يسمح بذلك.

Heartbeat .135

استخدم Heartbeat لتحديد ما إذا كان Partner لا يزال Online فعليًا.

لا تعتبر Partner Online للأبد بسبب آخر Status مخزن.

Scheduled Jobs .136

أنشئ Scheduler لمعالجة:

- .Scheduled services
- .Document expiry
- .Promotion expiry
- .Timeouts
- .Notification reminders

Job Timeline .137

كل Job يمتلك Timeline واضحًا.

مثلاً:

requested 12:00
assigned 12:01
partner arrived 12:10
started 12:15
completed 12:45

مع Admin events و System events.

Job Event History .138

لا تعتمد فقط على Current Status.

احتفظ بتاريخ Events.

Search .139

Admin search يدعم:

- .Job ID
- .Customer phone

- .Partner phone
- .Vehicle plate
- .Payment ID
- .Ticket

مع Permissions.

140. Fraud/Risk Architecture

جهاز Risk Engine قابلاً للتطوير.

Signals مثل:

- .Excessive cancellations
- .Promo abuse
- .Multiple accounts
- .Impossible GPS movement
- .Repeated failed payments
- .Unusual referral behavior

في V1 يمكن أن يبدأ بقواعد Rule-based.

141. Blocking

أنشئ:

User restrictions
Partner restrictions
Device restrictions

بما يتناسب مع سياسات المنصة.

مع Reason و Audit.

Admin RBAC .142

أدوار نموذجية:

SUPER_ADMIN
OPERATIONS
DISPATCHER
CUSTOMER_SUPPORT
PARTNER_SUPPORT
FINANCE
MARKETING
ANALYST

Super Admin فقط يحصل على أعلى الصلاحيات.

Sensitive Admin Actions .143

عمليات مثل:

- .Refund
- .Manual wallet adjustment
- .Partner suspension
- .Pricing changes

يجب أن تحتاج صلاحية محددة وتدخل Audit.

Wallet Adjustments .144

أي تعديل يدوي للرصد يجب:

- .Reason
- .Reference
- .Admin
- .Timestamp
- .Ledger entry

ولا يوجد:

... = UPDATE balance

مباشرة من لوحة الإدارة.

Receipts .145

أنشئ Receipt لكل عملية مدفوعة.

تحتوي:

- .Job number
- .Customer
- .Date
- .Service
- .Breakdown
- .Payment method
- .Total

حسب المتطلبات القانونية المحلية يمكن إضافة معلومات ضريبية مستقبلاً.

Admin Reports .146

تقارير قابلة للتصفية حسب:

- .Date
- .Zone
- .Service
- .Partner
- .Payment method

وتصدير CSV/XLSX عند الحاجة.

Future Food Module .147

يجب عدم تنفيذه الآن إذا كان خارج MVP، لكن Architecture يجب أن تسمح بإضافة:

Merchant
Branch
Menu
Product
Modifier
Cart
FoodOrder

مع إعادة استخدام:

- .Customer
- .Payments
- .Courier
- .Dispatch
- .Tracking
- .Promotions
- .Notifications
- .Support

Future Merchant Module .148

جهاز Domain boundaries بحيث تتم إضافة:

MERCHANT

دون تعديل Customer/Partner core بشكل فوضوي.

API Modules .149

Backend Modules الأساسية:

AuthModule
UsersModule
CustomersModule

PartnersModule
VehiclesModule
ServicesModule
JobsModule
DispatchModule
TrackingModule
PricingModule
ZonesModule
PaymentsModule
WalletModule
LedgerModule
PromotionsModule
ReferralsModule
NotificationsModule
ChatModule
MediaModule
RatingsModule
SupportModule
DisputesModule
AdminModule
AuditModule
AnalyticsModule
ConfigModule
RiskModule

Database Entities .150

صمم ERD كاملاً على الأقل للجداول التالية:

users
user_sessions
otp_requests

customer_profiles

partner_profiles
partner_roles
partner_skills
partner_documents
partner_availability

vehicles
vehicle_documents

service_types
service_categories
service_subcategories
service_options

service_zones
zone_service_rules

jobs
job_stops
job_media
job_assignments
job_events
job_tracking_points

service_quotes
service_quote_items
service_quote_revisions

pricing_rules
pricing_snapshots

payments
payment_attempts
refunds

wallets
ledger_accounts
ledger_entries
ledger_transactions

promo_codes
promo_redemptions

referral_codes
referral_rewards

reviews

chats
chat_members

messages

notifications
notification_templates

support_tickets
support_messages

disputes

admin_roles
admin_permissions

audit_logs

feature_flags

system_configs

قم بتحسين النموذج إذا وجدت طريقة أكثر احترافية، لكن لا تحذف Domain أساسي.

Database Documentation .151

لكل Table:

وثق:

- .Purpose
- .PK
- .FK
- .Indexes
- .Constraints
- .Data retention
- .Sensitive fields

Naming .152

استخدم Naming Convention واحدة في كامل المشروع.

تجنب:

userId
user_id
userid

في نفس الطبقة بطريقة عشوائية.

Code Quality .153

طبق:

- .SOLID
- DRY بدون over-abstraction
- .Clear boundaries
- .Dependency inversion
- .Small testable services

ممنوع God Classes.

No Business Logic in Controllers .154

:Controllers

- .Validation
- .Authorization entry
- .Call service
- .Map response

Business Logic داخل Domain/Application services.

No Business Logic in UI .155

Mobile UI لا تحسب Fare النهائي.

لا تحدد Commission.

لا تقرر Eligibility.

كل ذلك Server-side.

156. Shared Contract

استخدم DTO contracts واضحة.

مع Validation في Backend.

ولا تتق بال-Client حتى لو كان تطبيقنا الرسمي.

157. Documentation

أنشئ:

README.md
ARCHITECTURE.md
DATABASE.md
API.md
SECURITY.md
DEPLOYMENT.md
TESTING.md
OPERATIONS.md

158. Setup

يجب أن يستطيع Developer جديد تشغيل المشروع بخطوات واضحة.

مثلاً:

clone
env setup
docker compose
migrations
seed
run

159. Docker Local Environment

وفّر الخدمات اللازمة محليًا:

- PostgreSQL/PostGIS
 - Redis
 - Backend
 - Object-storage emulator عند الحاجة.
-

160. Seed Data

أنشئ Seed Development فقط.

يشمل:

- Service categories
- Vehicle types
- Zones example
- Admin
- Customer
- Driver
- Technician

لكن تأكد أن Seed development لا يعمل تلقائيًا في Production.

161. Demo Scenarios

جهز سيناريوهات اختبار قابلة للتكرار:

Ride

.Customer + Driver

Delivery

.Customer + Courier

Service

.Customer + Plumber

Design System .162

TAMAM يجب أن يمتلك Design System موحداً.

صفات التصميم:

- .Modern
- .Clean
- .Fast
- .Trustworthy
- .Simple
- .Arabic-first
- .Professional

Branding .163

استخدم الاسم:

TAMAM

والعربية:

تمام

اجعل هوية الواجهة قابلة للتعديل بواسطة Theme Tokens.
لا تنشر ألواناً Hardcoded في مئات الملفات.

UX Principle .164

الطلب يجب أن يتم بأقل عدد خطوات ممكن.
لا تجبر المستخدم على إدخال معلومات لا نحتاجها.

Maps UX .165

أهم الشاشات يجب أن تحتوي:

- Map
- Bottom Sheet
- واضح Pickup
- واضح Destination
- Current status

ولا تجعل الخريطة مزدحمة.

Privacy UX .166

عند طلب Location permission:

اشرح للمستخدم لماذا يحتاجها التطبيق.

Partner background location يجب طلبه فقط عندما يكون له سبب تشغيلي واضح ووفق متطلبات أنظمة الهاتف.

Registration UX .167

:Customer registration

.Phone → OTP → Name → Ready

لا تجعل عملية التسجيل طويلة.

Partner Registration .168

:Multi-step Partner onboarding يمكن أن يكون

Personal info

Role

Skills

Documents

Vehicle if needed

Service zone

Review

مع حفظ التقدم.

Partner Approval .169

Partner لا يستطيع استقبال طلبات قبل:

APPROVED

Dynamic Forms .170

Service Categories المستقبلية قد تحتاج Fields مختلفة.

لذلك أنشئ Architecture تسمح بـ Dynamic Service Request Fields بدل إعادة برمجة App لكل Service جديدة، مع أنواع حقول آمنة ومحددة مسبقاً.

171. AI Ready Architecture

لا تجعل AI جزءًا إلزاميًا من Core V1.

لكن جهاز Module يمكن إضافته لاحقًا:

AIServiceClassifier

يمكنه تحليل:

- .Text
- .Image

ثم اقتراح:

- .Category
- .Priority
- .Provider skill

لكن نتيجة AI ليست Authority نهائية.

172. Search Service

Customer يمكنه البحث:

سباك
تسريب مياه
كهربائي
مكيف

Search يجب أن يعمل بالعربية بشكل جيد.

173. Favorite Services

احفظ:

- .Recent
- .Favorites

لتحسين تجربة الاستخدام.

174. Customer Order History

:Filters

- All
- Ride
- Delivery
- Services
- Completed
- Cancelled

175. Reorder

يمكن إعادة طلب خدمة مشابهة اعتمادًا على Job سابق مع السماح للمستخدم بمراجعة البيانات قبل الإنشاء.

176. Partner Job History

يعرض:

- .Completed
- .Cancelled
- .Earnings
- .Date
- .Customer rating

177. Configuration Safety

عند تعديل Admin لقيمة حساسة مثل:

commission=100%

يجب وجود Validation تمنع القيم غير المنطقية.

Feature Rollout .178

Feature Flags يمكن أن تدعم مستقبلاً:

- .Global
 - .Zone
 - .Percentage rollout
 - .Specific users
-

Error Handling .179

كل External Service يجب أن يكون له:

- .Timeout
- .Retry policy
- .Circuit-breaker-ready design
- .Error mapping

لا تجعل API call معطلة بلا نهاية.

Maps Provider Abstraction .180

لا تربط كل Business Logic بمزود خرائط واحد.

أنشئ Interface مثل:

MapsProvider

يدعم:

geocode
reverseGeocode
route
distanceMatrix
placeSearch

حتى يمكن تغيير Provider مستقبلاً.

Payments Provider Abstraction .181

بنفس الطريقة:

PaymentProvider

SMS Provider Abstraction .182

SmsProvider

Storage Provider Abstraction .183

StorageProvider

Notification Provider Abstraction .184

لمنع Vendor Lock-in غير الضروري.

API Rate Limits .185

ضع Policies مختلفة حسب:

- .Login
- .OTP
- .Search
- .Job creation
- .Messages
- .Tracking

لا تستخدم Limit واحدًا للجميع.

Request Trace .186

كل API Request يحصل على:

requestId

للتشخيص.

Mobile Crash Reporting .187

جهاز Crash/Error Reporting abstraction.

مع عدم إرسال Secrets أو بيانات حساسة بشكل غير مقصود.

No Silent Failures .188

ممنوع:

`{}` (catch(error

يجب:

- .Handle
 - .Log appropriately
 - .Report to caller
 - Retry إذا كان مناسباً.
-

Quality Gates .189

قبل اعتبار Feature مكتملة يجب أن:

- compile
 - lint
 - type-check
 - tests pass
 - security checks pass
-

Definition of Done .190

الميزة غير مكتملة حتى:

- UI موجودة.
 - API موجود.
 - DB موجودة عند الحاجة.
 - Permissions مطبقة.
 - Validation مطبق.
 - Error states موجودة.
 - Tests موجودة.
 - Documentation محدثة.
 - Mobile responsive/usable.
 - Arabic RTL يعمل.
 - لا توجد Console errors.
 - لا توجد Broken actions.
-

191. لا تستخدم Fake Completion

لا تقل:

Implemented

إذا كانت الوظيفة مجرد UI.

حدد بدقة:

Implemented

Partially Implemented

Blocked

Not Implemented

192. Development Order

نفذ المشروع بهذا الترتيب المنطقي:

Phase 1

.Architecture + ERD + Repository + infrastructure

Phase 2

.Auth + Users + RBAC

Phase 3

.Services + Zones + Partners + Vehicles

Phase 4

.Universal Job Engine

Phase 5

.Pricing

Phase 6

.Dispatch

Phase 7

.Real-time tracking

Phase 8

.Customer App core

Phase 9

.Partner App core

Phase 10

.Ride flow

Phase 11

.Delivery flow

Phase 12

.Home service + Quote

Phase 13

.Payments + Wallet + Ledger

Phase 14

.Notifications + Chat

Phase 15

.Admin Dashboard

Phase 16

.Support + Disputes

Phase 17

.Analytics + Audit

Phase 18

.Security hardening

Phase 19

.Automated E2E

Phase 20

.Performance + Production readiness

193. بعد كل Phase

نفذ:

1. Build
2. Lint
3. Tests
4. Fix errors
5. Update documentation
6. Git-ready checkpoint

لا تنتقل للمرحلة التالية مع Errors أساسية غير معالجة.

194. Security Review قبل الإطلاق

نفذ مراجعة تشمل على الأقل:

- .Authentication
- .Authorization
- .IDOR
- .Injection
- .XSS

- .CSRF where relevant
- .CORS
- .SSRF
- .File upload
- .Rate limiting
- .Secret exposure
- .Token handling
- .WebSocket authorization
- .Payment duplication
- .Business logic abuse
- .Admin privilege escalation

195. Dependency Review

استخدم Dependencies موثوقة وضرورية فقط.

لا تضيف Package جديدًا إذا كانت فائدته بسيطة ويمكن تنفيذها بسهولة بدون زيادة Supply Chain Risk.

196. Dependency Pinning

استخدم Lockfiles.

وتأكد من أن CI يستخدم Dependency installation deterministic.

197. Production Checklist

لا تعتبر TAMAM جاهزًا للنشر إلا بعد:

- All migrations tested
- All E2E critical flows passed
- Backups tested
- Admin RBAC tested
- Payment idempotency tested
- Dispatch race test passed
- Tracking authorization tested

Secrets checked
Debug mode disabled
Production logging configured
Monitoring configured
Rate limiting configured
HTTPS configured

Expected Final Deliverables .198

في نهاية العمل يجب أن أحصل على:

Source Code

كامل.

Customer App

قابل للتشغيل.

Partner App

قابل للتشغيل.

Admin Dashboard

قابلة للتشغيل.

Backend

قابل للتشغيل.

Database migrations

كاملة.

Automated tests

كاملة للمسارات الحرجة.

API Documentation

كاملة.

ER Diagram

واضحة.

.Deployment guide

.Security guide

.Environment example

.Seed data

.Architecture documentation

199. تقرير نهائي

في نهاية التنفيذ أنشئ:

FINAL_IMPLEMENTATION_REPORT.md

ويحتوي:

- ما تم تنفيذه.
- .Modules
- .Architecture
- .Database
- .Security
- .Tests
- .Test results
- .Known limitations

- Disabled future features
 - Production checklist
 - Deployment steps
-

200. ممنوع في النسخة النهائية

ممنوع ترك:

TODO
FIXME
TEMP
HACK
fake data
mock API
demo-only implementation
hard-coded authentication
hard-coded prices
hard-coded admin permissions
unprotected admin route

في أي مسار Production أساسي.

201. قاعدة مهمة عند اكتشاف مشكلة

إذا وجدت مشكلة معمارية أثناء التنفيذ:

لا تبين Workaround فوقها.

قم بـ:

Identify root cause
Fix root cause
Update tests
Verify related modules
Document change

202. قاعدة عدم كسر النظام

بعد أي تعديل:

نفذ Regression Tests على:

Authentication
Job creation
Dispatch
Tracking
Pricing
Payment
Wallet
Permissions

حسب الأجزاء المتأثرة.

203. عدم حذف Features

عند تحسين Feature موجودة:

لا تحذف Feature أخرى تعمل بدون سبب موثق.

204. Database Safety

قبل Migration destructive:

- Analyze impact
 - Preserve data
 - Create migration strategy
 - Provide rollback where technically possible
-

205. Production Mindset

تعامل مع TAMAM وكأنه سيخدم:

100 مستخدم،

ثم:

10,000،

ثم:

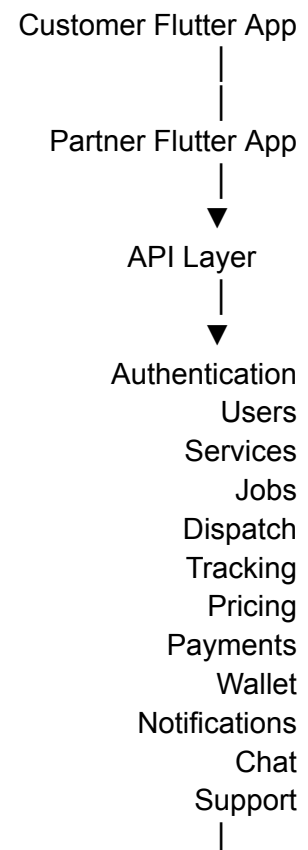
100,000+.

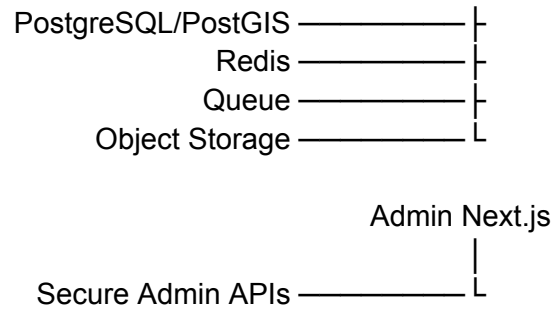
لكن لا تدخل في Premature Microservices.

صمم حدود Domains جيدة واجعل Scaling ممكنًا تدريجيًا.

Architecture Target .206

الشكل المستهدف:





207. Future Scalability

اجعل High-load domains قابلة للفصل مستقبلاً مثل:

Tracking Service
Dispatch Service
Notifications Service
Payments Service
Chat Service

لكن لا تحولها إلى Microservices قبل وجود سبب تشغيلي حقيقي.

208. الهدف النهائي

أريد TAMAM منصة حقيقية تنافس من ناحية تجربة المستخدم والاستقرار والأمان التطبيقات الكبيرة، لكن بهوية محلية بسيطة وسهلة.

يجب أن يشعر العميل أن:

- الطلب سهل.
- السعر واضح.
- مقدم الخدمة معروف.
- موقعه واضح.
- حالة الطلب واضحة.
- الدعم موجود.
- الدفع موثوق.

ويشعر Partner أن:

- الطلبات واضحة.
- الأرباح واضحة.
- الملاحه واضحة.
- لا توجد أخطاء مالية.
- لا تضيق الطلبات.

وتستطيع الإدارة:

- التحكم في العمليات.
- مراقبة الطلبات لحظيًا.
- إدارة الأسعار.
- إدارة المناطق.
- إدارة السائقين والفنيين.
- حل النزاعات.
- مراجعة الأموال.
- إضافة الخدمات بدون إعادة بناء التطبيق.

209. تعليمات التنفيذ النهائية

ابدأ أولاً بمراجعة هذا المستند كاملاً.

بعد ذلك:

1. أنشئ Architecture Specification.
2. أنشئ ERD.
3. حدد Domain Boundaries.
4. أنشئ Repository structure.
5. أنشئ Implementation Roadmap.
6. ابدأ التنفيذ الفعلي.
7. لا تتوقف عند إنشاء Skeleton.
8. نفذ Modules واحدة تلو الأخرى.
9. شغل الاختبارات باستمرار.
10. أصلح Root Causes وليس الأعراض.
11. لا تتجاوز أخطاء Compilation.
12. لا تتجاهل Failed Tests.
13. لا تعتبر UI مساوية لتنفيذ Feature.
14. تحقق من Integration الحقيقي بين التطبيقات والBackend.
15. تحقق من صلاحيات كل Endpoint.
16. تحقق من جميع العمليات الحسابية المالية.
17. تحقق من حالات Race Condition.
18. تحقق من جميع Job State transitions.

19. نفذ End-to-End testing للمسارات الثلاثة الأساسية.

20. أعد مراجعة النظام بالكامل قبل إعلان الاكتمال.

210. Acceptance Gate النهائي

قبل إعلان:

TAMAM READY

يجب تنفيذ سيناريو كامل حقيقي لكل من:

RIDE

Customer → Request → Driver Accept → Tracking → Arrival → Start → Finish → Payment →
.Rating

DELIVERY

Customer → Package Request → Courier Accept → Pickup Verification → Tracking → Delivery
.Verification → Payment → Completion

SERVICE

Customer → Select Service → Upload Problem → Technician Accept → Arrival → Inspection →
.Quote → Customer Approval → Work → Completion → Payment → Rating

ويجب أن تنجح السيناريوهات من:

Frontend → API → Database → Real-Time → Financial Ledger → Admin Dashboard

بدون تعديل يدوي لقاعدة البيانات.

إذا فشل أي جزء:

لا تعتبر النظام مكتملاً.

قم بإصلاح المشكلة ثم أعد الاختبار.

PROJECT NAME

TAMAM

PRODUCT VISION

.One app. Any local service. Done. TAMAM

ابدأ تنفيذ المشروع وفق هذه المواصفات باعتبارها الحد الأدنى للجودة، وليس مجرد اقتراحات.